

QueueCure - Engineering Thought Process & System Design

1. Executive Summary

QueueCure was designed to address a critical inefficiency in the Indian healthcare system: the black box of clinic waiting rooms. Traditional paper token systems create crowded, contagious environments, overwhelm receptionist staff, and provide zero visibility to patients regarding their actual wait time. QueueCure V2 solves this by introducing a highly scalable, real-time, event-driven architecture that calculates dynamic wait times based on historical doctor efficiency.

2. Problem Analysis

During our initial research phase, we identified three core stakeholders and their pain points:

1. **The Patient:** Experiences high anxiety due to uncertain wait times. They are forced to stay in the waiting room to avoid missing their turn.
2. **The Receptionist:** Spends up to 50% of their operational time answering the question, "How much longer?"
3. **The Doctor:** Suffers from workflow interruptions when patients step out and are not present when called.

3. Architectural Decisions & Tradeoffs

3.1. Separated Monorepo Architecture (Next.js + Express)

We chose to decouple the frontend and backend rather than building a monolith (e.g., Next.js API routes).

Reasoning: Real-time WebSocket management (Socket.IO) is notoriously difficult to scale in serverless environments like Vercel. By running a dedicated Node/Express server, we maintain persistent WebSocket connections required for instant queue synchronization.

Tradeoff: Increased deployment complexity (requires a containerized backend rather than a simple Vercel push).

3.2. Real-Time Protocol: WebSockets vs. Server-Sent Events (SSE)

We evaluated SSE for broadcasting queue updates. While SSE is unidirectional and perfectly suited for a read-only TV display, the Receptionist and Doctor dashboards require bidirectional communication (e.g., adding patients, calling the next token).

Decision: Socket.IO was chosen for its automatic reconnection logic, built-in fallback to HTTP long-polling, and "Rooms" feature, which allowed us to instantly implement multi-clinic tenancy (`io.to(clinicId)`).

3.3. Database: MongoDB (NoSQL) vs. PostgreSQL (SQL)

Given the rigid schema of patients and clinics, SQL would be a natural fit. However, queue systems require rapid reads and flexible event logging.

Decision: We selected MongoDB. The QueueEvent collection benefits heavily from NoSQL's flexible schema (Mixed details field), allowing us to log disparate events (Patient Added, Priority Changed, Queue Paused) without complex SQL table joins or JSONB querying overhead.

4. The Wait Time Engine 2.0 Logic

The core intellectual property of QueueCure is the Wait Time Engine.

V1 Limitations (Static Multiplier)

In V1, wait time was simply Patients Ahead * 10 minutes. This failed the reality test: if the current patient has been in the doctor's office for 45 minutes, the next patient's estimate would still incorrectly read "10 minutes."

V2 Innovation (Dynamic Rolling Average)

We completely rewrote the engine:

1. **Calculate Rolling Average:** The system queries the ConsultationLog to find the average consultation duration over the last 10 visits.
2. **Account for Elapsed Time:** If the doctor is currently seeing Patient A, the system calculates Elapsed Time = Now - PatientA.calledAt.
3. **The Formula:** $EWT = \text{Max}(0, \text{RollingAvg} - \text{ElapsedCurrent}) + ((\text{PatientsAhead}) * \text{RollingAvg})$. This ensures that every second the doctor spends over the average instantly pushes back the wait time for everyone else in the queue, providing unprecedented accuracy to the patient.

5. Concurrency Handling & Atomic Operations

A massive challenge in queue systems is Race Conditions. If two receptionists click "Call Next Patient" simultaneously, the system could skip a patient or assign two patients the "IN_CONSULTATION" status.

Solution: We utilize MongoDB's findOneAndUpdate atomic operations. The backend explicitly searches for { status: 'WAITING' }, sorts by Priority/Token, and atomically transitions the exact document to IN_CONSULTATION. This guarantees thread-safety at the database layer without requiring heavy Redis locking mechanisms.

6. Priority Queue Implementation

In a clinical setting, Emergencies must bypass standard queues.

Instead of maintaining separate arrays, we implemented a Priority weighting system: EMERGENCY (3), PRIORITY (2), NORMAL (1).

When calculating the queue array, the backend sorts descending by Priority Weight, and then ascending by Token Number. This approach is highly efficient for typical clinic daily volumes (< 500 patients) and avoids complex aggregate pipelines.

7. Event Sourcing for Future AI

Instead of simply updating rows, we implemented an Event Sourcing pattern. Every mutation creates a QueueEvent.

Business Impact: In the future, we can feed this immutable ledger into Machine Learning models to predict clinic peak hours based on weather, day of the week, or seasonality. It also provides an irrefutable audit log for clinic administrators.

8. Scalability & Multi-Tenancy Strategy

QueueCure V2 introduces clinicId across all schemas.

To scale from 1 clinic to 10,000 clinics:

1. **Database:** We can shard MongoDB by clinicId.
2. **WebSockets:** We will introduce the `@socket.io/redis-adapter`. When Clinic A's reception connects to Server Node 1, and Clinic A's TV connects to Server Node 2, Redis Pub/Sub ensures the `QUEUE_UPDATED` event traverses the nodes and reaches the TV instantly.

9. Security & Privacy Considerations

- Tracking links (`/track/[id]`) use an 8-byte cryptographically secure random hexadecimal string (`queueId`) instead of the sequential token number or MongoDB ObjectID. This prevents malicious actors from enumerating the URL and viewing other patients' names.
- Offline Fallback: The Zustand store actively monitors the socket connection. If the hospital Wi-Fi drops, the frontend automatically falls back to a 10-second REST API polling mechanism, ensuring the TV screen is never stale.

10. Lessons Learned & Future Roadmap

Lessons Learned: We quickly realized that strict time slot appointments do not work in India. The system had to be built around a *rolling token queue* rather than a calendar.

Roadmap:

1. WhatsApp API Integration: Send a WhatsApp message to the patient when there are exactly 2 people ahead of them.
2. Doctor Analytics Export: PDF generation for clinic owners to evaluate doctor efficiency.
3. Multi-Doctor Support: Allowing multiple doctors in a single clinic to pull from a unified or specialized queue.

Conclusion

QueueCure is not just a hackathon project; it is a meticulously engineered SaaS platform. By combining predictive algorithms, real-time WebSocket architecture, and atomic database operations, we have built a product that is genuinely ready for production deployment.